Facts as Hyperedges and Discourse Nodes: A Unified Narrative Hypergraph for Graph Retrieval-Augmented Generation

Anonymous Author(s)

ABSTRACT

Graph-based retrieval-augmented generation (RAG) has a large language model (LLM) turn a corpus into a knowledge graph and retrieves along the graph's links to ground the LLM's answers. Most such indices connect two pieces of knowledge only through an entity they both mention, whether they store triples, *n*-ary facts, or propositions, or they fuse knowledge into summaries that the LLM writes. Many questions instead follow relations between *statements*. Suppose an insured man understated his age when he applied for life insurance and died years later. Whether his beneficiary is paid depends on four clauses in three sections of the policy: the two-year limit on contesting the application, its exception for the date of birth, the misstatement-of-age clause, and the eligibility age limit. The last two decide the answer but share no entity, so no entity hop connects them. A community summary can relate only the clauses its cluster happens to contain, and it paraphrases the very conditions that decide the answer. Discourse and event graphs relate statements, and hypergraphs keep the entities of each statement together. The answer path above needs both at once. We present the Unified Narrative Hypergraph (UNH), an index that combines the two over the same facts. Every atomic fact extracted from the corpus is both a hyperedge over the entities that take part in it and a node in a discourse graph of typed relations between facts, such as EXCEPTION_TO, DEPENDS_ON, and CAUSES. An LLM decides these relations, each is stored as a labelled edge rather than written into a summary, and every fact stays linked to the passage that states it. To find relations across documents without comparing every pair, UNH matches the follow-up questions each fact raises against the questions other facts answer. It clusters facts by this affinity and judges each cluster in one LLM call, so the number of calls grows linearly with the corpus. At query time, one LLM call predicts which relation types the answer should follow. A beam search prefers those relations to generic entity hops, and the reader receives short chains of source facts joined by their relation labels. We compare UNH with six graph-based RAG systems on HotpotQA, MuSiQue, and QuALITY under a protocol that fixes the LLM, embedding model, reader, and judge. We test whether both structures are needed by evaluating hypergraph-only and discourse-only views of the same index.

Abid: Add the headline result sentence once the UNH runs are scored.

CCS CONCEPTS

• **Information systems** → **Retrieval models and ranking**; • **Computing methodologies** → **Knowledge representation and reasoning**; *Information extraction.*

KEYWORDS

Retrieval-augmented generation, knowledge graph construction, knowledge hypergraphs, discourse relations, multi-hop question answering

1 INTRODUCTION

Retrieval-augmented generation (RAG) grounds a large language model (LLM) in an external corpus [17]. Graph-based RAG goes one step further. An LLM first turns the corpus into a knowledge graph, and retrieval then follows the graph's links to collect evidence that is related to other evidence, not only to the question [6, 10, 11, 34]. These graphs are a new kind of knowledge graph on the Web: built automatically from text, specific to one corpus, and read by a machine. What such a graph can connect determines what the machine reading it can be told.

A motivating example. Insurance policies are published on the Web, and the people they cover, or agents acting for them, ask questions about them. Consider a beneficiary's question about a term-life policy (Figure 1): "*My father applied for this policy saying he was 58, but he was actually 61. He died six years later, long after the two-year contestability period. Will I receive the death benefit?*" The question is hard because the clause it names is not the clause that decides it. The answer, *no; the insurer refunds the premiums instead*, needs four clauses from three sections of the policy.[1]

- The general provisions state that after two years the insurer treats the answers in the application as true (f_1). However, this rule does not apply to the insured's date of birth (f_2).
- The misstatement-of-age clause has two branches. If the insured would still have been eligible at the correct age, the premium is corrected and the benefit is reduced by the underpaid amount (f_3). If not, the policy is treated as never having been in force, and the premiums are refunded (f_4).
- Which branch applies is decided pages earlier, by the eligibility section: applications are accepted only up to six months past the 60th birthday (f_5).

At 61, the father was too old to qualify, so f_4 applies, not f_3. A reader who sees the misstatement clause without the age limit will pick the wrong branch and answer that the benefit is paid, minus the underpaid premiums.

Abid: Add the source of the sample policy (TD 20-Year Term Life sample policy package) in the camera-ready version; keep it generic for review.

Why entity links are not enough. The answer path alternates between two kinds of connection, as multi-hop questions about

[1] Clauses abridged from a public sample of a 20-year term-life policy.

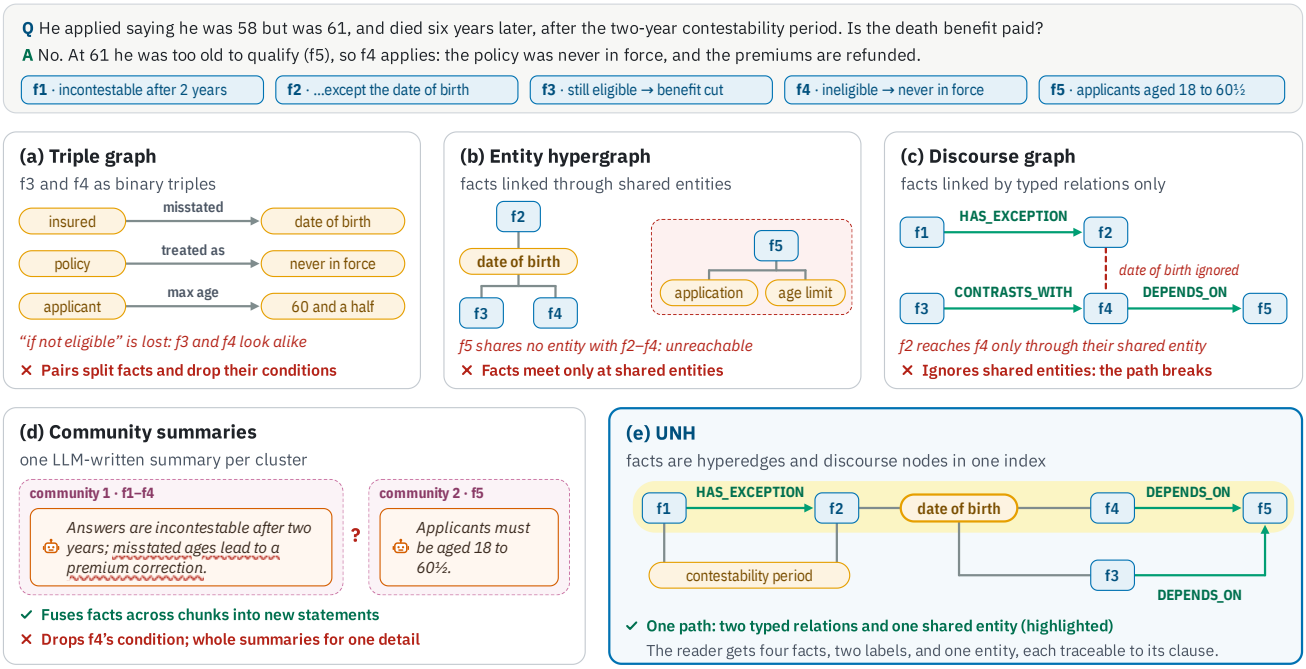


Figure 1: *Will the beneficiary be paid?* The insured stated an age of 58 when he applied but was 61, and he died six years later. The answer joins four clauses from three sections of a term-life policy (top). They are the two-year contestability rule (f_1), its exception for the date of birth (f_2), the misstatement-of-age branch for an insured who would not have been eligible (f_4), and the eligibility age limit (f_5). f_3 is the other branch, for an insured who would still have been eligible. (a) A triple graph splits f_3 and f_4 into binary triples and loses the condition that tells them apart. (b) An entity hypergraph links f_2, f_3, and f_4 through *date of birth*, but f_5 shares no entity with them. (c) A discourse graph links facts by typed relations but ignores shared entities, so the path breaks between f_2 and f_4. (d) Community summaries fuse facts across chunks, but they generalize away f_4's condition and give the reader whole summaries for one detail; the dependence between the two clusters is written in neither. (e) UNH uses both edge families: f_1 HAS_EXCEPTION f_2, f_2 reaches f_4 through *date of birth*, and f_4 DEPENDS_ON f_5.

events do [18]. Two of its steps follow relations between statements: f_2 is an exception to f_1, and f_4 depends on f_5 for what "eligible" means. One step follows an entity: f_2 and f_4 both concern the insured's *date of birth*, which appears in only three of the policy's 251 sentences. Entity-based indices capture the entity step and miss the relations (Figure 1a–b). A triple graph [6, 10] also splits each clause into pairs, so f_3 and f_4 both become (*insured*, *misstated*, *date of birth*) and lose the condition that tells them apart. A triple graph and a hypergraph of n-ary facts [22] connect f_1 and f_2 through the contestability period without saying that one is an exception to the other. They connect f_2 to both branches through *date of birth*. They do not connect either branch to f_5: the misstatement clause speaks of a date of birth, a policy, and premiums, while the eligibility rule speaks of applications and an age limit. Nothing in either graph says that the eligibility rule decides which branch applies. Dense retrieval returns the contestability and misstatement clauses, which share the question's words. It ranks the eligibility rule, which shares little more than "age", far below them. A discourse graph [35, 54] has the opposite gap (Figure 1c). It relates f_1 to f_2 and f_4 to f_5, but it has no entities, so nothing leads from the exception for the date of birth to the clause about a misstated date of birth.

Why not community summaries? A natural remedy is to let the LLM write the connections down (Figure 1d). Microsoft GraphRAG [6], RAPTOR [37], and HyGRAG [57] cluster the corpus or its graph and write a summary of each cluster. A summary can fuse what several passages say and state what none of them states alone. One could even prompt the summaries to spell out every causal, temporal, and conditional relation among their members. Three problems remain, and a better prompt solves none of them.

- *A summary can only relate what its cluster contains.* Clusters are formed from shared entities or from embedding similarity. The misstatement clause therefore falls with the other general provisions, and the eligibility rule with the application requirements. The dependence the question needs lies *between* the two clusters and is written in neither summary.
- *A summary is fixed before any question is asked.* It must choose which of the many relations among its members to

state, at one granularity, in a paragraph the reader receives whole. It cannot be cut down to the one chain a question needs, or chained with another cluster's summary.

- *A summary paraphrases.* In a contract, the scope of a clause is often the answer. A summary that compresses the misstatement clause to "misstated ages lead to a premium correction" drops the branch that decides this question.

Summaries remain the right tool for questions about a corpus as a whole [6]. For questions that follow a chain of specific statements, the connections must be stored where retrieval can select them one at a time, between facts that the reader can check against their source. The fusion then happens at query time, by the reader, over exactly the facts the question needs.

The gap. Entity-centric indices, the dominant design in graph RAG, therefore cannot represent the relation between two statements: it is absent, reduced to a hub entity, or dissolved into generated text. This has two consequences. First, *dependent facts that share no informative entity cannot be found from each other*, although rules and their exceptions, conditions and the definitions they rely on, causes and effects, and claims and their rebuttals are routinely stated with different terms and in different places. Second, *a step from one fact to the next says nothing about how they relate*, so retrieval cannot prefer the exception to a rule over the many other facts that share its entities. MEQA [18] documents the same gap for question answering. Multi-hop benchmarks are built around bridges between entities, whereas questions about events, such as "*Who died before Al-Monitor reported it online?*", must alternate between entity–event and event–event relations. Given the documents, GPT-3.5 with chain-of-thought reaches 0.29 F1 on such questions. Adding an entity graph raises it only to 0.33, while adding a graph with explicit relations between events raises it to 0.49. Neither ingredient of a remedy is new. The Semantic Web has long treated statements as objects in their own right. N-ary relation patterns [31], Wikidata's qualified statements [44], RDF-star [12], and nanopublications [7] all give a statement an identity to which further assertions can attach. Hypergraph and proposition indices make n-ary facts the unit of retrieval [22, 45]. Discourse theory and event knowledge graphs relate statements by cause, condition, contrast, and time [25, 35, 54], and event-centric RAG retrieves along such relations [21, 52]. What the example shows is that an answer path needs both at once, over the same facts. It alternates between relations, which connect statements that share no entity, and entities, which connect statements that no stated relation links. In existing RAG indices one of the two is weak. N-ary indices connect facts only through entities. Event and discourse approaches find relations within one chunk or among passages already retrieved, or traverse them with an agent over several LLM calls (Section 2).

Abid: Strengthen with one number: the share of gold reasoning chains in QuALITY and MuSiQue whose consecutive supporting facts share no entity, or share only an entity mentioned by more than 5% of facts, e.g., measured on 100 annotated questions.

Our approach. We introduce the **Unified Narrative Hypergraph (UNH)**, an index that combines a hypergraph and a discourse graph over the same facts. Every atomic fact extracted from the corpus plays two roles. It is a *hyperedge* over the entities that take part in it, as in a hypergraph. It is also a *node* in a discourse graph, a directed graph of relations between facts, where each relation has a short type label such as EXCEPTION_TO, DEPENDS_ON, CAUSES, or CONTRADICTS. Every fact keeps a link to the passage that states it. In Figure 1e, the date-of-birth exception f_2 is linked to the contestability rule f_1 by EXCEPTION_TO. The ineligibility branch f_4 is linked to the age limit f_5 by DEPENDS_ON, although the two share no entity. A retrieval path can therefore enter at the clause the question names (f_1) and follow a typed relation to its exception (f_2). It can then cross to the misstatement clause through the specific entity *date of birth* (f_4) and follow a second typed relation to the age limit (f_5). This path alternates the two edge families, and it exists in neither the hypergraph nor the discourse graph alone. The reader receives the chain, with each clause traceable to its section, and concludes that the policy was never in force.

Two problems stand between this representation and a practical index. The first is *which facts to relate.* Asking an LLM about every pair of facts is quadratic in corpus size. Embedding similarity alone misses exactly the pairs we want, because the misstatement clause and the eligibility rule have little in common. UNH annotates each fact with the questions it answers and the follow-up questions it raises. It then scores a candidate pair by how well the questions one fact raises match those the other answers. Here, the ineligibility branch f_4 raises *what are the age limits for this policy?*, and the eligibility rule f_5 answers *what ages can apply for this policy?*. Facts are clustered by this affinity, and one LLM call decides the relations within each cluster, so the number of calls grows linearly with the corpus. Unlike a summary, the result of the call is a set of labelled edges. Edges found in different clusters can be chained by retrieval, and each edge can be selected on its own. The second problem is *how retrieval should use the relations.* A single LLM call turns the question into a plan that names the relation types the answer path should follow (here HAS_EXCEPTION and DEPENDS_ON). A beam search prefers edges whose labels match the plan. The reader then receives short, deduplicated chains of facts joined by their relation labels and by the entities they share.

We compare UNH with six graph-based RAG systems: HippoRAG 2 [10], LogicRAG [3], MemGraphRAG [47], HyperGraphRAG [22], Microsoft GraphRAG [6], and HyGRAG [57]. The benchmarks are multi-hop questions over Wikipedia (HotpotQA [51], MuSiQue [41]) and long narratives (QuALITY [33]). Every system uses the same LLM, embedding model, reader prompt, and judge, so differences come from indexing and retrieval alone. Our contributions are:

- **Representation.** UNH, a knowledge-graph model for RAG that combines an n-ary fact hypergraph and a discourse graph over the same atomic facts, with passage-level provenance. Neither structure is new. Each is an edge-restricted view of UNH, and the combination admits answer paths that neither view contains (Section 3.1).
- **Construction.** A pipeline that finds typed relations between facts across documents with a number of LLM calls linear in corpus size, using question-based affinity and

community-level judgment that outputs edges rather than summaries (Section 3.2).
- **Retrieval.** A procedure with one LLM call per question that seeds from facts and entities jointly, follows the relation types predicted from the question, and returns grounded fact chains (Section 3.3).
- **Evaluation.** A controlled comparison of seven systems that reports accuracy together with construction tokens and reader context. It includes an ablation over the hypergraph-only and discourse-only views of the same index, which tests whether both structures are needed (Section 4).

Abid: Add one sentence with the headline findings after the UNH runs.

2 RELATED WORK (CLAUDE VERSION)

We group prior work by what its index can connect, because that determines which questions retrieval can follow. For each group we state what it does and what it leaves out. Table 1 summarizes the comparison as capabilities.

Entity-centric graph RAG.. The dominant design has an LLM extract entity–relation triples from each passage and merge them into one knowledge graph [1, 16, 28, 53]. Microsoft GraphRAG [6] partitions the graph into communities and pre-generates a report for each. LightRAG [8] retrieves entities and relations through keywords. HippoRAG [9] and HippoRAG 2 [10] run Personalized PageRank from the question's entities, and HippoRAG 2 adds passage nodes and LLM filtering of retrieved triples. MemGraphRAG [47] shares a global memory between extraction agents to keep the schema consistent and to resolve conflicting triples. Iterative reasoners [23, 24, 38] and learned retrievers [26] walk the same triples. Systematic comparisons find that the graph pays off mainly when a question requires connecting evidence [48, 58]. These methods leave out two things. A fact with more than two participants is split into triples that no longer record which participants belong to the same event. Facts are connected only through entity nodes, so a hub entity offers all of its facts as equal neighbours.

N-ary facts and propositions. A second line keeps a fact's participants together. HyperGraphRAG [22] stores each *n*-ary fact as a hyperedge over its entities. HyperRAG [20] learns to reason over such facts, HOLMES [32] builds a hyper-relational graph per query, and Cog-RAG [14] pairs an entity hypergraph with a hypergraph of chunk themes. Other methods change the retrieval unit to the atomic proposition [4, 27]. GraphReader [19] attaches atomic facts to key elements of a long document. PropRAG [45] treats propositions as hyperedges and beam-searches over proposition paths. SiReRAG [55] groups propositions that share entities and summarizes each group. AtomicRAG [13] and ToPG [5] link atomic facts to entities and passages. These methods recover *who* takes part in a fact. The connection between two facts, however, is still co-membership of an entity, a theme, or a chunk. It is untyped, and it does not exist when two facts share none.

Events, causality, and discourse. The methods closest to ours relate statements directly. ASER [54], presented at this conference, built an eventuality graph whose edges are discourse relations of the Penn Discourse Treebank [35], mined from explicit connectives. It is a commonsense resource, not an index of a given corpus. EventRAG [52] extracts events with typed relations such as CAUSE and PRECEDE. Its relations come from the extraction call of one chunk, under-connected events are expanded with links from shared participants or from the LLM's own knowledge, and an agent retrieves in several LLM rounds. E^{2}RAG [56] and DyG-RAG [39] link events by co-mention, shared entities, or temporal proximity, without relation types. CausalRAG [46] builds causal edges between concepts and asks an LLM to find causal paths at query time. Disco-RAG [21] parses rhetorical relations within and between the passages retrieved for a question and uses them to plan the answer. UNH builds on this line: its relations between facts are discourse relations in the same sense, and we do not claim the discourse graph as new. What these methods do not provide together is what a retrieval index needs. The typed relations should be (i) found across documents, (ii) between grounded statements rather than concepts or event names, (iii) built once over the whole corpus, and (iv) used to decide *what* to retrieve. They should also be kept alongside the entities of each statement. A query-time structure over already retrieved passages cannot recover evidence that similarity search missed.

Generated summaries and query-time structure. Some indices fuse knowledge across passages by writing new text. RAPTOR [37] recursively summarizes clusters of chunks, NodeRAG [49] adds summary nodes to a heterogeneous graph, and HyGRAG [57] clusters a hybrid chunk–entity graph and writes summaries that fuse context and relations. Agent memories [36, 50] link or rewrite stored notes with an LLM. Other methods build structure per question. IRCoT [42] interleaves retrieval with reasoning, and Plan*RAG [43] and LogicRAG [3] decompose the question into a graph of sub-questions; LogicRAG also keeps a rolling LLM summary of the evidence. Summaries can state what no single passage states, which suits questions about a corpus as a whole [6]. For questions that follow a chain of statements, they have three limits regardless of prompt (Section 1). They relate only what their cluster contains, they fix at index time which relations to state, and they paraphrase conditions the reader cannot trace to a source. Query-time structure is discarded after each question, and its LLM cost recurs on every question (Section 4.5).

Statements as knowledge objects. Knowledge representation on the Web has long given statements an identity. The W3C pattern for *n*-ary relations [31] introduces a node for each relation instance, Wikidata attaches qualifiers and references to statements [44], RDF-star lets a triple be the subject of another triple [12], and nanopublications bundle an assertion with its provenance [7]. Discourse theory supplies the relations between statements: cause, contrast, elaboration, and temporal succession [25, 35]. Both ideas are established. UNH combines them in one index that an LLM builds from text. A fact is a statement node with its participants (the *n*-ary pattern) and its source passage (its provenance), and typed relations hold between facts. We do not fix a relation inventory; labels are open-vocabulary, produced

under constraints (Section 3.2), and may join facts from different documents.

The gaps UNH closes. Table 1 distills four gaps. **(G1) Connectivity:** entity-centric and n-ary indices connect facts only through shared entities, and the methods with relations between statements find them within one chunk or among already retrieved passages. UNH finds typed relations across the whole corpus, including between facts that share no entity. **(G2) Guidance:** no corpus-level index chooses its next hop by the kind of relation a question asks for. UNH predicts relation types from the question and follows edges with matching labels. **(G3) Grounding:** methods that connect distant evidence do so through generated summaries or the LLM's own knowledge, which fix the connections before the question is known and paraphrase the facts they connect. UNH stores each connection as a labelled edge that retrieval selects per question, and gives the reader extracted facts, the entities that connect them, and the labels of their relations, with each fact traceable to its passage. **(G4) Cost:** methods that build structure per question pay for it on every question. UNH builds its structure once with a number of LLM calls linear in the corpus and needs one LLM call per question.

Abid: Check the MemGraphRAG and PropRAG rows against their retrieval code (number of query-time LLM calls), and decide whether to add CausalRAG and LightRAG rows.

3 METHODOLOGY

Given a corpus $\mathcal{D} = \{d_1, \ldots, d_n\}$ of documents and a question, UNH returns a small set of evidence chains extracted from $\mathcal{D}$, from which a reader LLM answers the question. The work is split into two phases. An offline phase builds the index $\mathcal{G}$ once per corpus (Section 3.2, Figure 2). An online phase retrieves from $\mathcal{G}$ for each question (Section 3.3, Figure 3). Below, $\cos(a, b)$ is the cosine similarity between the embeddings of two texts a and b; a set of texts is embedded as one concatenated string.

Running example. The text follows the policy question of Figure 1 from start to end; Figure 2 follows the same example; Figure 3 illustrates retrieval on a small open-domain example. The example *starts* with the beneficiary's question: the insured stated an age of 58 but was 61, and died six years after applying, so is the death benefit paid? It *ends* with the reader's context and its answer: the policy is treated as never in force, and the premiums are refunded. Table 2 shows an illustrative index for this example: facts, entities, and relations of the kind UNH builds, chosen to make the steps concrete. Each step below states which of them it would produce or use.

3.1 Index Structure

Graph structure. A UNH index is a graph

$$\mathcal{G} = (C, F, V, E_{CF}, E_{FV}, E_{FF})$$

with three kinds of nodes and three kinds of edges (Figure 1e).

Chunks C are the source passages of the corpus. Each carries its text and a one-sentence summary.

Facts F are atomic, self-contained propositions stated in a chunk. Each fact is a tuple

$$f = \big(\underbrace{s}_{\text{sentence}}, \underbrace{T}_{\text{topics}}, \underbrace{A}_{\text{answers}}, \underbrace{U}_{\text{follow-ups}} \big),$$

where s is the fact's sentence with every reference resolved, T is a set of topics, A is the set of questions the fact answers, and U is the set of follow-up questions a reader of the fact would ask next. For the ineligibility branch f_4 (Table 2), s is "If a policy was issued using an incorrect date of birth and the insured would not have been eligible at the correct age, the policy is treated as never in force and all premiums are refunded". Its topics are $T = \{$misstatement of age, eligibility$\}$, and it answers $A = \{$What happens if the insured was too old to qualify at the true age?$\}$. It raises $U = \{$What are the age limits for this policy?$\}$, a question that f_4 cannot answer itself and that the eligibility rule f_5, in another section, does answer.

Entities V each carry a canonical name n and a *micro-role* ρ, a short phrase stating what the entity is or does in its passage. For example, *date of birth* has the role "the insured person's birth date, which sets their age for eligibility and premiums".

The three edge sets record provenance, participation, and dependence. The provenance relation $E_{CF} \subseteq C \times F$ links each fact to the chunks that state it. The participation relation $E_{FV} \subseteq F \times V$ links each fact to the entities that take part in it. Its participant set $V(f)$ makes each fact a hyperedge of any arity over V. The relation set E_{FF} holds directed, labelled edges (f_i, f_j, ℓ, r) between facts. The label ℓ is a short type naming how the two propositions relate (CAUSES, PRECEDES, EXCEPTION_TO, DEPENDS_ON, CONTRADICTS), and $r \in [0, 1]$ is the strength the LLM assigns to the relation. Edges come in pairs that name both directions, such as $(f_2, f_1,$ EXCEPTION_TO$)$ and $(f_1, f_2,$ HAS_EXCEPTION$)$.

The dual role of facts. Facts are the only nodes that are connected both to entities and to each other. Through E_{FV}, a fact is a hyperedge: it keeps together all participants of one statement, which a triple graph would split across several edges. Through E_{FF}, a fact is a node of a discourse graph: it is linked to the facts it causes, precedes, qualifies, depends on, or contradicts, whether or not they share a participant. The two edge families support complementary moves. Participation leads from a fact to the entities it involves and from an entity to every fact that involves it, as from f_2 through *date of birth* to f_4. Relations lead from a fact to the facts it depends on or that depend on it, and say how, as from f_4 to f_5. Both kinds of node and both kinds of edge reach the reader. A retrieved path is shown as its fact sentences, the entities on it with their names and micro-roles, and the labels of the relations between its facts. Chunks record provenance and are never traversed. The question fields A and U are used only to decide which facts to compare during construction.

Edge-restricted views. Removing one edge family gives a familiar index. Without E_{FF}, what remains is the hypergraph $\mathcal{G}_H = (V, \{V(f) : f \in F\})$, an n-ary fact index of the kind used by hypergraph RAG [22]. Without V and E_{FV}, what remains is the discourse graph $\mathcal{G}_D = (F, E_{FF})$, a labelled graph over facts of the kind built

Table 1: Capabilities of graph-based RAG methods. N-ary facts: one element of the index holds all participants of a fact. Typed fact relations: the index stores relations between facts (or events or passages), labelled by the kind of dependence. No-shared-entity links: the index can connect two facts from different passages that share no entity. Relation-guided hops: retrieval chooses its next step by the type of relation the question asks for. Grounded context: the reader sees only text from the corpus, each piece traceable to its source; relation labels count as types, not text. Single-pass query: at most one LLM call before the reader, over an index built once. ✓: yes; ○: partly (see note); ✗: no; –: not applicable, because the method builds no corpus index.

Method	What the index connects			What retrieval can do		
	N-ary facts	Typed fact relations	No-shared-entity links	Relation-guided hops	Grounded context	Single-pass query
MS GraphRAG [6]	✗	✗	✗	✗	○[a]	✓
HippoRAG 2 [10]	✗	✗	✗	✗	✓	✓
MemGraphRAG [47]	✗	✗	✗	✗	✓	✓
HyperGraphRAG [22]	✓	✗	✗	✗	○[a]	✓
PropRAG [45]	✓	✗	✗	✗	✓	✓
HyGRAG [57]	✗	✗	○[b]	✗	○[a]	✓
EventRAG [52]	✓	✓	○[c]	○[d]	○[e]	✗
Disco-RAG [21]	✗	○[f]	○[f]	✗	○[a]	✗
LogicRAG [3]	–	–	–	–	✗	✗
UNH (ours)	✓	✓	✓	✓	✓	✓

[a] Corpus text mixed with LLM-written entity descriptions, community reports, summaries, or plans. [b] Only inside generated cluster summaries. [c] Relations are extracted within one chunk; cross-document links come from shared participants or from the LLM's own knowledge. [d] An LLM agent decides each step over several calls. [e] The graph may contain knowledge the LLM adds from its own parameters. [f] Typed relations between passages, built per question over the passages already retrieved.

Table 2: An illustrative UNH index for the running example: five facts from three sections of the policy, abridged. Entities omit the hub *policy*. f_4 and f_5 share no entity; they are connected only by the relation found in Step 8. The follow-up question of f_4 is answered by f_5, and that of f_5 by f_3 and f_4. This asymmetric match is what puts f_4 and f_5 in one community.

	Fact sentence s	Entities $V(f)$	Answers A	Raises U
f_1	After two years, the insurer treats the answers given as evidence of insurability as true, except for fraud.	evidence of insurability; two-year contestability period	Can the insurer still challenge the application after two years?	Are any answers excluded from the two-year rule?
f_2	The two-year rule does not apply to the insured person's date of birth.	two-year contestability period; **date of birth**	Does the two-year rule cover a misstated date of birth?	What happens if the date of birth was misstated?
f_3	If the date of birth was misstated and the insured would still have been eligible, the premium is corrected and the benefit is reduced by the underpaid amount.	**date of birth**; insured person; premium; death benefit	What happens if the age was misstated but the insured still qualified?	How is the underpaid premium calculated?
f_4	If the date of birth was misstated and the insured would not have been eligible at the correct age, the policy is treated as never in force and all premiums are refunded.	**date of birth**; insured person; premium	What happens if the insured was too old to qualify at the true age?	What are the age limits for this policy?
f_5	Applications are accepted only from people aged 18 to six months past their 60th birthday.	application; age limit	What ages can apply for this policy?	What happens if an applicant misstates their age?

Relations: f_1 HAS_EXCEPTION f_2 and f_3 CONTRASTS_WITH f_4, within a chunk (Step 4). f_3 DEPENDS_ON f_5 and f_4 DEPENDS_ON f_5, across sections that share no entity (Step 8). **Entity link:** *date of birth* joins f_2, f_3, and f_4, and occurs in only three of the policy's 251 sentences.

from discourse and event relations [35, 52, 54]. Both views are established index structures. The full index connects more than either view, and this is what the combination adds.

Proposition 3.1 (Edge-restricted views). *There exist a UNH index $\mathcal{G}$ and facts f, f' such that f' is reachable from f in $\mathcal{G}$ but in neither $\mathcal{G}_H$ nor $\mathcal{G}_D$.*

Proof. Let $F = \{f_1, f_2, f_3\}$ and $V = \{v_1, v_2\}$, with $V(f_1) = V(f_2) = \{v_1\}$, $V(f_3) = \{v_2\}$, and one relation edge from f_2 to f_3. In $\mathcal{G}$, the path $f_1 \to v_1 \to f_2 \to f_3$ reaches f_3. In $\mathcal{G}_H$, f_3 shares no entity with f_1 or f_2; in $\mathcal{G}_D$, f_1 has no edge. Hence f_3 is unreachable from f_1 in both views. □

The running example is an instance of the proposition. Its answer path $f_1 \rightarrow f_2 \rightarrow$ *date of birth* $\rightarrow f_4 \rightarrow f_5$ uses both families. In $\mathcal{G}_H$, f_5 is unreachable from f_1, because it shares no entity with f_1, f_2, or f_4. In $\mathcal{G}_D$, f_4 is unreachable from f_1, because no relation leads from f_2 to the misstatement clause; only the shared date of birth does. Proposition 3.1 also underlies our ablation (Section 4.3). Restricting traversal to E_{FV} or to E_{FF} gives hypergraph-only and discourse-only variants over the *same* facts and entities, so their differences from the full system come from the edges available at traversal time, not from extraction.

3.2 Index Construction

Construction turns $\mathcal{D}$ into $\mathcal{G}$ in eight steps (Figure 2). Steps 1–4 run independently for every chunk. Steps 5–8 work on the merged graph. Every LLM call fills a typed output record [15], without chain-of-thought. The reader later sees three kinds of LLM output: fact sentences, entity names with their micro-roles, and relation labels. The others (chunk summaries, topics, and questions) only guide construction and retrieval. Short passages (HotpotQA, MuSiQue) are one chunk each, prefixed with their title; long documents (QuALITY, policies) are split into overlapping token windows.

Step 1: Chunk summary and entities. One LLM call per chunk returns a one-sentence summary of the chunk and its entity inventory: canonical names, with pronouns and aliases resolved, each with a micro-role. The summary is later shown to the LLM as context when facts from different chunks are compared, and is never shown to the reader. The inventory fixes the chunk's entities before any fact is extracted, so all facts of a chunk refer to the same entity objects. For the chunk with the general provisions, the inventory contains *evidence of insurability* ("answers given in the application, on which the insurer relies"), *two-year contestability period* ("time after which those answers can no longer be challenged"), and *date of birth*.

Step 2: Atomic facts. A second call extracts the chunk's facts as sentences that are *atomic* (one claim, event, rule, or state), *self-contained* (every reference resolved), and *grounded* (stated in the chunk). Self-containment lets a fact be read without its passage [4, 27]. The policy's "If the insured person is not eligible for insurance, all coverages will be considered never to have been in force" becomes f_4, which restates that the age was misstated. Atomicity separates the two branches of the misstatement clause into f_3 and f_4. A passage would return them together with the premium-adjustment rules around them. As separate facts, each branch can receive its own relation to the eligibility rule, and retrieval can return both without the surrounding clauses.

Step 3: Fact annotation. Facts are annotated in batches. Given the chunk, a batch of facts, and the inventory names, one call returns for each fact its topics, the questions it answers, the follow-up questions it raises, and the inventory entities that take part in it. Each returned name is matched to the inventory, exactly or else to the most similar name under character n-gram TF–IDF, and becomes a participation edge. Choosing participants from the inventory, rather than extracting names afresh, keeps a chunk's facts attached to shared entity objects.

The answered and follow-up questions are what make relations across sections and documents findable. They describe what a fact is *for* rather than what it *says*, and the two sets of a fact differ. The answers of a fact describe what it settles; its follow-ups describe what it leaves open, which another fact must settle. The ineligibility branch f_4 settles what happens to a policy whose insured was too old to qualify, but it leaves open how old is too old. The eligibility rule f_5, pages earlier, settles exactly that (Table 2).

Step 4: Relations within a chunk. For each chunk with at least three facts, one call receives the chunk summary and the numbered facts. It returns every related pair with a forward label, a backward label, and a strength in $[0, 1]$. Labels are constrained in two ways. First, a label must name a relation *between* two propositions (CAUSES, PRECEDES, EXCEPTION_TO, CONTRADICTS), not an attribute of one of them (HAS_AMOUNT), which would duplicate participation. Second, the backward label must name the inverse (EXCEPTION_TO/HAS_EXCEPTION). Pairs with zero strength are discarded. In the running example, this step labels f_1 HAS_EXCEPTION f_2 and f_3 CONTRASTS_WITH f_4. Triple-based systems store a free-text description on each relation [6, 8]. A UNH edge stores only its label and strength: the facts already carry the content, and the label is enough to steer traversal and to tell the reader how two facts connect.

Step 5: Entity resolution and fact consolidation. The chunk subgraphs are merged, and duplicate entities are resolved in passes of increasing cost. The first pass is exact match after normalization (Unicode NFKC, lower-casing, singularization). The second is semantic hashing at high similarity.[2] The third handles the remaining entities with a nearest-neighbour graph that weighs name similarity above role similarity. Leiden [40] partitions this graph, and small clusters are packed together. One LLM call per cluster then decides which members denote the same entity and assigns a canonical name and role; edges of duplicates are redirected to the canonical node. In the running example, the *date of birth* of the general provisions, the *correct date of birth* of the misstatement clause, and the *Date of Birth of Insured Person* in the policy schedule merge into one entity. This merge creates the entity step from f_2 to f_4. The micro-role keeps namesakes apart: "Jordan" the river and "Jordan" the basketball player share a name but not a role. Duplicate facts stated by several chunks are then merged the same way, over a graph of fact similarity. The result is one fact whose provenance is the union of its sources, following the conflict taxonomy of MemGraphRAG [47].

Abid: Fact consolidation (`fact_dedup`) defaults to on in `evaluation_scripts/unh/main.py`, but NOTES.md says it was switched off for the runs. Confirm which setting the reported runs used, and delete the last two sentences if it was off.

Step 6: Multi-view fact affinity. Relations between facts of different chunks supply the connectivity that shared entities cannot. Asking an LLM about all pairs, however, is quadratic. Our design choice is to score candidate pairs with several cheap signals, each capturing a different reason two facts may be related, and to send only

[2] SemHash, https://github.com/MinishLab/semhash.

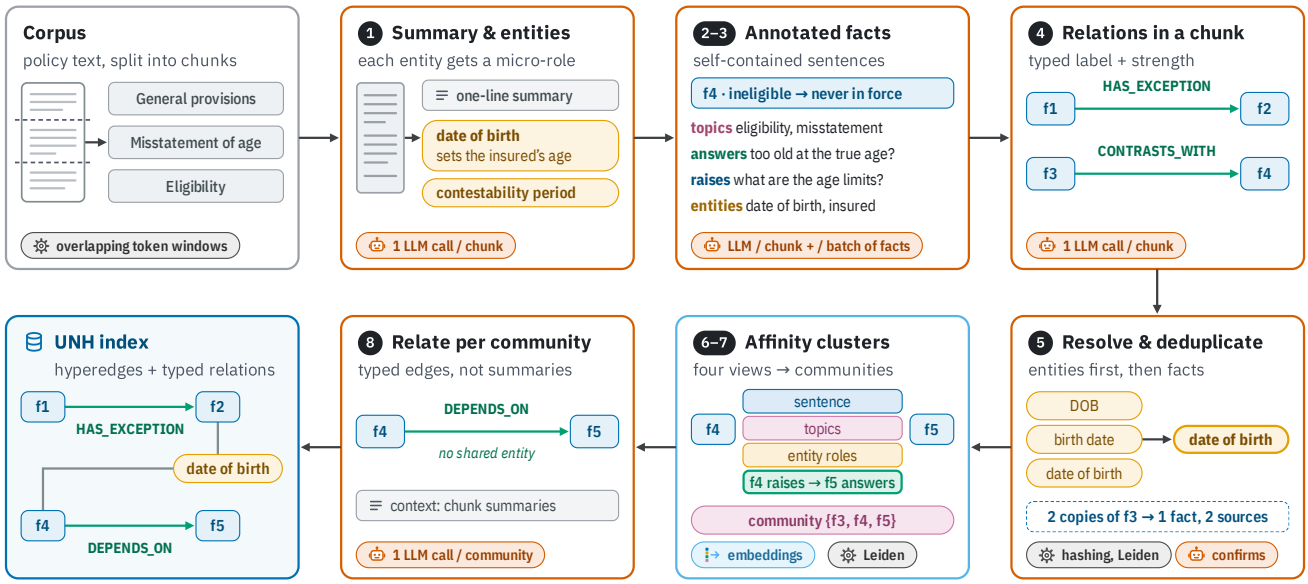


Figure 2: Index construction on the running example. Dark badges give the steps of Section 3.2; the bottom badges show what does the work (orange: LLM calls; blue: embeddings; grey: algorithms). The top row runs in parallel for every chunk. The bottom row works on the merged graph: it resolves entities and merges duplicate facts, clusters facts by affinity, and relates the facts of each community. Here, step 8 adds f_4 DEPENDS_ON f_5 between two sections that share no entity.

promising groups to the LLM. The affinity of f_j as a partner of f_i is the sum of four similarities:

$$\mathrm{aff}(f_i, f_j) = \underbrace{\cos(s_i, s_j)}_{\text{sentence}} + \underbrace{\cos(T_i, T_j)}_{\text{topics}} + \underbrace{\cos(R_i, R_j)}_{\text{roles}} + \underbrace{\cos(U_i, A_j)}_{\text{raises} \to \text{answers}},$$

where R_i is the set of micro-roles of f_i's participants. The first three signals find facts that are about the same thing. The fourth is asymmetric and finds facts that complete each other. It compares the follow-up questions f_i raises with the questions f_j answers. It is therefore high when f_j supplies what a reader of f_i would look for next, which is the bridge of a multi-hop question, even when the two sentences share no words or entities. For f_4 and f_5, the sentence and role signals are low, but "What are the age limits for this policy?" and "What ages can apply for this policy?" are close paraphrases. The topic signal adds to this through the shared topic *eligibility*.

Step 7: Fact clustering. Each fact keeps as candidates the k facts with the highest affinity to which it is not yet related. The union of these neighbourhoods is a sparse weighted graph with at most $k|F|$ edges. Hierarchical Leiden [40] partitions it into communities of at most M facts. A community may span chunks and documents. It is a unit of comparison, not of summarization, and no text is generated for it. Because the affinity rewards facts that complete each other, a community can join the misstatement branches f_3 and f_4 with the eligibility rule f_5. A cluster built from shared entities would keep them apart.

Step 8: Relations within each community. For each community, one call receives its facts as a numbered list. Each fact is preceded by the summary of its chunk, for example "`[Context: misstatement of age and sex at birth]` If the date of birth was misstated and the insured would not have been eligible…" and "`[Context: eligibility to apply]` Applications are accepted only from…". The call returns the related pairs with labels and strengths under the constraints of Step 4. The summary tells apart similar sentences about different subjects and is not stored. Judging communities rather than pairs has three effects. The LLM sees each candidate alongside its other likely partners and can choose among them. It can relate facts that were grouped transitively rather than directly. The number of calls is bounded by the number of communities, which grows linearly with the number of facts. In the running example, this step adds f_4 DEPENDS_ON f_5 and f_3 DEPENDS_ON f_5, between sections that share no entity.

This step differs from community summarization [6, 57] in its output, not in its input. A summary writes the relations among a community's facts into one paragraph. Step 8 writes each relation as a separate labelled edge between two facts. Edges from different communities meet at shared facts and entities, so retrieval can chain them across community boundaries. They can be selected one at a time, so a question receives only the relations on its path. And they leave the facts unchanged, so the reader still sees each clause with its conditions.

Cost. Steps 1, 2, and 4 issue one call per chunk, step 3 one per batch of facts, step 5 one per cluster of candidate duplicates, and step 8 one per community. Every term is linear in the number of chunks or facts, so construction cost grows linearly with the corpus (Section 4.5).

3.3 Retrieval

Retrieval turns a question into a small set of paths in six steps (Figure 3). Only the first calls an LLM; every later step compares embeddings or applies a cross-encoder. Two similarities between a set of plan items P and a text x recur below. The best match is $\cos_{\max}(P, x) = \max_{p \in P} \cos(p, x)$, and the average match is $\cos_{\text{avg}}(P, x) = \frac{1}{|P|} \sum_{p \in P} \cos(p, x)$. Both are 0 when P is empty.

Step 1: Query planning. One LLM call turns the question into a plan with five fields:

- the topics P_T;
- the named entities P_N, restricted to proper nouns and identifiers;
- entity roles and types P_R;
- the relation labels P_L that the answer path should follow;
- short phrases P_S describing the connection sought.

Each field holds one to three items or is left empty rather than filled with invented content. The question itself is not rewritten. For the running example, the plan is P_T = {misstatement of age, contestability, eligibility}, P_R = {insured person, date of birth}, P_L = {HAS_EXCEPTION, DEPENDS_ON}, and P_S = {what decides whether a misstated age voids the policy}. P_N is empty, because the question names no proper noun. The plan is the only point at which an LLM interprets the question. It is what lets retrieval prefer, among the many edges out of a fact, the exception and the dependence that this question needs.

Step 2: Joint scoring of facts and entities. Facts and entities compete in one candidate pool:

$$\text{score}(f) = w_s \cos(\text{question}, s_f) + w_T \frac{1}{|P_T|} \sum_{p \in P_T} \cos_{\max}(T_f, p),$$
$$\text{score}(v) = w_n \cos_{\max}(P_N, n_v) + w_\rho \cos_{\max}(P_R, \rho_v).$$

A fact scores high when its sentence resembles the question and its topics match the plan's topics; the topic term averages, over the plan's topics, the similarity of each to the fact's closest topic. When the plan or the fact has no topics, w_T moves to the sentence term. An entity scores high when its name matches a named entity of the plan or its micro-role matches a role in the plan. A question that names its subject therefore enters the index through the entity. A question that describes a situation without naming its participants enters through the fact. Here, the contestability rule f_1 matches the question's sentence, and *date of birth* matches a role in the plan.

Abid: Claude: A question should still be able to enter through entities because of entity roles.

Step 3: Diverse seeds. Seeds are selected from the pool by Maximal Marginal Relevance [2], which adds at each step the candidate u that maximizes $\lambda \, \text{score}(u) - (1 - \lambda) \max_{u' \in \mathcal{S}} \cos(u, u')$, where $\mathcal{S}$ is the current seed set. A fact is represented by its sentence and an entity by its role. Without this step, the seeds of a multi-hop question tend to be paraphrases of one fact. In the running example, the two branches f_3 and f_4 are near-paraphrases, and so are the policy's other statements about misrepresentation. MMR keeps one of them and gives the remaining seed slots to f_1 and to *date of birth*.

Step 4: Label-guided beam search. From each seed, paths expand over participation edges in both directions and over relation edges, never revisiting a node. Each candidate step is scored by signals that depend on its edge type:

- **Relation step** $f_i \rightarrow f_j$ with label ℓ and strength r: $\alpha_L \cos_{\text{avg}}(P_L, \ell) + \alpha_S \cos_{\text{avg}}(P_S, \ell) + \alpha_r \, r$. The step is preferred when its label matches the relations and connections in the plan and the LLM judged it strong.
- **Fact-to-entity step** $f \rightarrow v$: $\cos_{\text{avg}}(P_R, \rho_v)$. The step is preferred when the entity's micro-role matches the roles in the plan.
- **Entity-to-fact step** $v \rightarrow f$: participation edges carry no label in the evaluated configuration, so all facts of an entity tie and the beam keeps up to B of them without preference. At a specific entity such as *date of birth*, which three facts mention, the beam keeps them all. At a hub entity, it keeps an arbitrary B, and the choice is left to the cross-encoder in Step 5.

At every step, each partial path is extended by its B best next steps, selected by MMR over the label embeddings so that they differ. No score is accumulated along a path; the steps only decide which paths are grown. A path ends when it contains D facts, when it has $2D$ nodes, or when no unvisited successor remains. In the running example, the search from f_1 prefers HAS_EXCEPTION to f_2 over f_1's other relations, because the label matches P_L. From f_2, it prefers *date of birth*, whose role matches P_R, to *two-year contestability period*. From *date of birth*, it keeps both branches f_3 and f_4. From each branch, it follows DEPENDS_ON to f_5. These steps produce two complete paths: $f_1 \rightarrow f_2 \rightarrow$ *date of birth* $\rightarrow f_4 \rightarrow f_5$ and the same path through f_3.

Step 5: Path ranking. Each completed path is serialized as alternating nodes and edges. Facts appear as quoted sentences and entities as "[name (micro-role)]". A relation appears as its label. A participation step appears as "(has ρ)" from fact to entity and "(participates in event)" from entity to fact. The answer path reads, abridged:

```
"After two years, the answers are treated as true"
  -> HAS_EXCEPTION -> "The two-year rule does not
     apply to the date of birth"
  -> (has …) -> [date of birth (sets the
     insured's age)] -> (participates in event)
  -> "If not eligible at the correct age, the policy
     was never in force; premiums are refunded"
  -> DEPENDS_ON -> "Applications are accepted only
     up to six months past the 60th birthday"
```

A cross-encoder [30] scores each serialized path against the question. A final MMR pass then selects P paths, trading this score against the similarity of the path embeddings. The cross-encoder reads the whole chain at once, so it judges whether the chain, not only its last fact, answers the question. The selection is by relevance and diversity alone; paths are not chosen for how much of the plan they cover.

Step 6: Context assembly. Paths kept for one question often share prefixes. The best path is emitted in full. Each remaining path that overlaps it contributes only its steps that have not yet been emitted, and a path that overlaps none starts a new group. The context is therefore a set of chains of fact sentences joined by relation labels

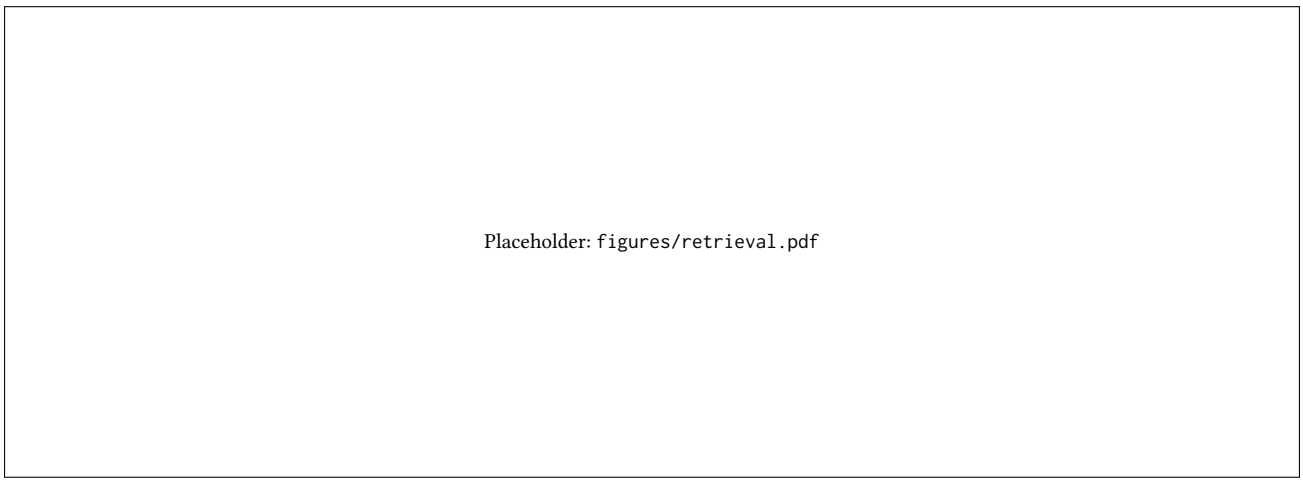


Figure 3: Retrieval, illustrated on the open-domain example of Figure 2 for "*Why did residents leave the old town?*". One LLM call plans the query. Facts and entities are scored in one pool and diverse seeds are selected. A beam search follows the relation labels the plan asks for: from the seed *residents* it reaches f_3, then f_2 by PRECEDED_BY, then f_1 by CAUSED_BY, although f_1 shares no entity with either. Paths are ranked by a cross-encoder, and the reader receives deduplicated fact chains.

and by entities with their micro-roles. Each fact appears once and can be traced to its chunk.

End of the running example. The context begins with the answer path of Step 5. The path through the other branch adds only its new part, "[date of birth (sets the insured's age)] $\rightarrow f_3 \rightarrow$ DEPENDS_ON $\rightarrow f_5$". The reader thus sees the rule the question names, its exception, both branches of the misstatement clause, and the age limit that decides between them. Since 61 exceeds the limit, f_4 applies: the policy was never in force, the death benefit is not paid, and the premiums are refunded. Every step of the reasoning is a fact or an edge the reader can check. Each component contributed one part of this context: atomic facts separated the branches, entity resolution created the date-of-birth link, question annotations and community judgment found the dependence on the age limit, and the plan and beam search followed both.

Cost. Retrieval issues one LLM call per question, for the plan. The rest consists of embedding lookups, a search over a sparse graph, and one cross-encoder pass over the candidate paths. Hyperparameters are listed in Appendix B.

4 EXPERIMENTS

We address the following research questions:

- **RQ1** Does UNH answer more accurately than existing graph-based RAG methods under a controlled protocol?
- **RQ2** Are both structures needed? Does each view of the index, the hypergraph alone and the discourse graph alone, fall short of their combination?
- **RQ3** Where do relations help most: in long narratives, where facts are linked by cause and time, or in encyclopedic multi-hop questions, where they are linked by names?
- **RQ4** How much context does UNH pass to the reader, and at what construction cost, relative to existing systems at their default settings?

4.1 Experimental Setup

Datasets. For multi-hop reasoning we use the HotpotQA [51] and MuSiQue [41] releases of HippoRAG 2 [10], also used by MemGraphRAG [47], LogicRAG [3], and LinearRAG [59]: 1,000 questions each over pooled corpora of 9,811 and 11,656 Wikipedia passages. All systems are compared on a fixed, seeded 250-question subset of each, stratified by question type (HotpotQA: 203 bridge, 47 comparison) or hop count (MuSiQue: 129 two-, 79 three-, 42 four-hop); full-set baseline results are in Appendix A. For reading comprehension we use QuALITY [33]: multiple-choice questions about stories and articles of about 5k tokens, whose HARD subset cannot be answered by skimming. Its answers hinge on causal, temporal, and motivational links between events rather than on distinct names. Following HyGRAG [57], each system indexes the 115 articles of the official development split (0.66M tokens) in one pooled index and answers all 2,086 questions (1,065 HARD).

Abid: Optional: add the insurance-contract benchmark (5 issued policies, 445 questions in five categories) as a further column group of Table 3 once it is run.

Corpora. The baselines index the full corpora. UNH currently indexes only the passages of the 250 evaluated questions (2,488 HotpotQA and 4,025 MuSiQue passages, the standard distractor setting), so its rows are not yet strictly comparable to the baseline rows.

Abid: Before submission, either build UNH over the full corpora (`UNH_CORPUS=full`) or rebuild the baselines on the subset corpora, and then delete this paragraph's caveat.

Baselines. We compare with **HippoRAG 2** [10] (triples and passages, Personalized PageRank), **LogicRAG** [3] (no pre-built graph; query-time sub-question DAG with a rolling summary), **MemGraphRAG** [47] (memory-guided multi-agent triple extraction), **HyperGraphRAG** [22] (n-ary hyperedges), **Microsoft**

GraphRAG [6] (version 3.2.0, local search), and **HyGRAG** [57] (hybrid chunk–entity hierarchy with fused summaries). Each system runs from its official code with its default settings, ingestion, and retrieval; we change only model endpoints. The reader receives the context each system's own answer step would use: the top-5 passages (HippoRAG 2), the top-5 256-token windows (MemGraphRAG), the final rolling summary after at most three rounds of top-5 retrieval (LogicRAG), the hybrid entity/hyperedge/source context (HyperGraphRAG), the local-search context with a 12k-token budget (GraphRAG), and one community summary with four chunks, five entities, and five relations (HyGRAG). On QuALITY, the two systems without a chunker of their own (HippoRAG 2 and LogicRAG) receive the 1,200/100-token chunks produced by HyGRAG's chunker; the others chunk the articles themselves.

Controlled protocol and metrics. Every system uses the same LLM for all internal calls (DeepSeek V4.1 Flash, reasoning mode off, provider-default temperature), the same embedding model (voyage-4-nano), and the same reader: a shared answer step that applies HippoRAG 2's one-shot QA prompt to the system's context (for QuALITY, a prompt that returns the option letter). Only the context differs across systems. We report exact match (EM) and token-level F1 over the gold answer and its aliases, and LLM-judged accuracy (LLM-Acc) with the judging prompt of LinearRAG [59], where the judge sees only the predicted and gold answers; for QuALITY we report the accuracy of the chosen option overall and on HARD questions. UNH is compared with each baseline on the same questions by a paired bootstrap (10,000 resamples, 95% intervals); at $n = 250$ one question is 0.4 points and a 95% interval on F1 spans roughly ±4–5 points. UNH uses `ms-marco-MiniLM-L6-v2` as its cross-encoder and the hyperparameters of Table **??**; like the baselines, its context is not truncated.

Abid: Confirm UNH chunking for QuALITY (600/50-token windows by default; the baselines received 1200/100-token chunks).

4.2 Main Results (RQ1)

Table 3 reports the results. Under the controlled protocol, HyperGraphRAG is the strongest baseline on both multi-hop datasets (tied with HippoRAG 2 on HotpotQA EM), and its margin is largest on MuSiQue (57.2 F1, 7.5 points above the next system), consistent with the argument that preserving n-ary facts helps when evidence must be combined. It also passes by far the longest context of any system to the reader (about 21.7k tokens; Section 4.5). HippoRAG 2 and LogicRAG follow on HotpotQA, and Microsoft GraphRAG on MuSiQue. MemGraphRAG ranks lower than in its own paper, which used a different LLM, embedding model, and metric, illustrating why a shared protocol is needed before published numbers are compared. Baseline rankings on the subsets match those on all 1,000 questions up to one adjacent swap per dataset (Appendix A).

Abid: Fill the UNH rows and paired-bootstrap intervals. Discuss: margin over HyperGraphRAG at a fraction of its context; whether the gain grows with hop count on MuSiQue; failures where seeds were correct but traversal was not.

4.3 Hypergraph View versus Discourse View (RQ2)

Our central claim is that a retrieval index needs both structures: relations between facts, stored as edges, and the entities of each fact. To test it, we evaluate the two views of the *same* index (Proposition 3.1) with the same seeds, ranking, and reader. **UNH-H** traverses only participation edges E_{FV}, so facts reach each other only through shared entities, as in hypergraph RAG. **UNH-D** traverses only discourse edges E_{FF}; entities can still serve as seeds but cannot be traversed through, as in a discourse or event graph. Because extraction and entity resolution are shared, the differences among UNH-H, UNH-D, and UNH (Table 3) come from the edges available at traversal time. UNH-D versus UNH-H compares the two established structures on equal footing. UNH versus both measures whether their combination retrieves evidence that neither view reaches alone.

Abid: Fill and discuss; report the share of retrieved paths that mix both edge families, and the share of relation steps between facts that share no entity.

Case study.

Abid: Trace one QuALITY HARD question end to end, in the style of Figure 1: the plan, the seeds, the path UNH follows (with its relation labels and source chunks), and the context of the strongest baseline for the same question. Pick a question whose path crosses a relation between facts that share no entity.

4.4 Effectiveness across Domains (RQ3)

Wikipedia multi-hop questions reward connecting short encyclopedic passages through named entities, which is the setting entity-centric indices are designed for. QuALITY tests the opposite regime: long narratives in which relevant facts are linked by causation, temporal order, and motivation more often than by distinct names. We expect the discourse edges to matter most on QuALITY's HARD questions, which require integrating evidence from several parts of a story. Among the baselines, HippoRAG 2 (82.7) and HyGRAG (81.7) lead, within about one standard error of each other (≈0.9 points at $n = 2{,}086$), followed by HyperGraphRAG (79.4) despite a context four times longer, and MemGraphRAG (74.8), whose 256-token windows give the shortest context. HARD questions keep this order and cost every system 6–8 points.

Abid: Fill LogicRAG, MS GraphRAG, and UNH on QuALITY and discuss; add further domains as column groups.

4.5 Reader Context and Construction Cost (RQ4)

Table 4 reports the LLM tokens consumed by index construction and the mean length of the context passed to the reader on the evaluated questions. Construction is paid once per corpus; context is paid on every query.

Construction. Construction cost varies by more than an order of magnitude. HippoRAG 2 and HyGRAG, which issue one

Table 3: Overall QA results. All systems share the LLM, embedding model, reader prompt, and judge. HotpotQA and MuSiQue: 250-question subsets; QuALITY: development set (HARD: hard subset). LLM: LLM-judged accuracy. UNH-H (hypergraph view) and UNH-D (discourse view) restrict traversal to participation edges and to relation edges between facts, respectively, on the same index as UNH. Bold: best; underline: second best (over the rows filled so far). QuALITY: LogicRAG and MS GraphRAG still running.

Method	Reading Comp.		Multi-Hop Reasoning					
	QuALITY		HotpotQA			MuSiQue		
	Acc	HARD	EM	F1	LLM	EM	F1	LLM
HippoRAG 2	**82.7**	**76.6**	**59.6**	71.9	80.4	36.0	47.1	52.0
LogicRAG	–	–	56.4	71.3	81.6	33.6	44.5	48.8
MemGraphRAG	74.8	67.0	50.4	61.4	68.4	30.4	38.8	41.6
HyperGraphRAG	79.4	71.7	**59.6**	**74.8**	**85.6**	**46.4**	**57.2**	**62.0**
MS GraphRAG	–	–	51.6	65.4	75.6	38.8	49.7	55.2
HyGRAG	81.7	75.8	56.4	69.3	79.2	32.0	40.4	44.0
UNH-H (hypergraph)	–	–	–	–	–	–	–	–
UNH-D (discourse)	–	–	–	–	–	–	–	–
UNH (ours)	–	–	–	–	–	–	–	–

Table 4: Construction cost and reader context. In/Out: construction LLM input and output tokens (millions); Ctx: mean reader-context tokens per question (`cl100k_base`; context only). Baselines index the full HotpotQA and MuSiQue corpora and the 115 QuALITY development articles; UNH currently indexes the subset corpora (Section 4.1). †LogicRAG builds no index; its context is a summary written by the LLM at query time.

Method	HotpotQA			MuSiQue			QuALITY		
	In	Out	Ctx	In	Out	Ctx	In	Out	Ctx
HippoRAG 2	9.25	3.59	634	10.50	3.75	664	1.86	0.39	5,328
LogicRAG†	0	0	66	0	0	115	0	0	–
MemGraphRAG	158.70	16.51	1,241	153.45	16.05	1,240	45.50	4.22	1,292
HyperGraphRAG	161.93	17.57	21,783	189.95	19.59	21,540	19.72	5.75	20,492
MS GraphRAG	47.37	21.40	9,087	49.33	22.43	8,898	–	–	–
HyGRAG	13.77	4.33	1,000	15.26	4.52	1,018	2.93	0.48	5,014
UNH (ours)	–	–	–	–	–	–	–	–	–

entity and one triple-extraction call per passage (plus community summaries for HyGRAG), use 12.8–19.8M tokens. Microsoft GraphRAG spends about two thirds of its 69–72M tokens on community reports. MemGraphRAG and HyperGraphRAG use 170–210M tokens; for MemGraphRAG, two thirds of the input goes to schema induction and most of the rest to conflict detection. On QuALITY the ordering is the same, but long articles make construction cheaper per corpus token for systems that call the LLM once per chunk: HyperGraphRAG spends 30 input tokens per corpus token (125 on HotpotQA) and HippoRAG 2 2.8 (7.1), while MemGraphRAG, whose 256-token windows do not grow, spends 69 (122).

Abid: Fill UNH from `construction_results.json` (per-stage split). The NOTES smoke test measured about 7.9k input / 4.8k output tokens per HotpotQA passage; since UNH indexes fewer passages, also report tokens per passage for all systems.

Reader context. Under their defaults, context lengths range from 634 tokens (HippoRAG 2's five passages) through about 1,000–1,250 (HyGRAG, MemGraphRAG) to 9k (GraphRAG) and 21.7k (HyperGraphRAG). The strongest baseline in Table 3 is also the one with the longest context, so accuracy and context length are hard to separate across systems. QuALITY preserves this ordering (MemGraphRAG 1.3k, HyGRAG 5.0k, HippoRAG 2 5.3k with its longer chunks, HyperGraphRAG 20.5k tokens). UNH returns at most $P = 20$ deduplicated chains of at most D facts each.

Abid: Fill the UNH context length and state the comparison with HyperGraphRAG.

The case of LogicRAG.. LogicRAG's final context is the shortest (66 and 115 tokens), but it measures something different. LogicRAG builds no index and does at query time the work other systems do offline: it retrieves five passages, decomposes the question when they do not suffice, retrieves five more passages per sub-problem, and after each round has the LLM rewrite a rolling summary; only

the last summary reaches the reader. On the MuSiQue subset it performs 1.3 additional rounds on average and reads about 11.5 passages (about 1,270 tokens of raw evidence, counting repeats) to produce its 115-token summary (HotpotQA: 0.4 rounds, 6.9 passages, about 810 tokens), and its query-time pipeline consumes about 2.9k LLM input and 0.6k output tokens per question (HotpotQA: 1.8k and 0.3k, averaged over all 1,000 questions, including its own answer and judge calls). Its short context is thus the output of query-time compression rather than of more selective retrieval, and its cost recurs on every question; it should be compared on the evidence it inspects, not on the length of its summary.

Caveat. Every system, including UNH, runs at its default context size; we did not search for the budget at which each performs best. A system with a long default context might keep most of its accuracy at a smaller budget, so our comparison characterizes the systems as released rather than their best accuracy–context trade-off.

5 DISCUSSION AND LIMITATIONS

Construction cost. UNH trades construction cost for structure: every relation is adjudicated by an LLM and several calls are issued per chunk. The number of calls is linear in corpus size, but the constant is larger than for extraction-only systems; label-only relation prompts and smaller annotation schemas reduce it.

Inferred relations. Facts are extracted from the corpus, but relations between facts of different documents are the LLM's judgment: the policy does not state that the eligibility rule decides which branch of the misstatement clause applies. UNH keeps the two apart. A relation stores only a label and a strength, both of its facts remain traceable to their passages, and the reader sees the label as a connector between quoted facts rather than as a claim of the corpus. Wrong relations can still mislead traversal; recording which community call produced each relation would make them auditable.

Abid: Optional: measure relation precision on a sample of 100 cross-chunk edges judged by hand.

Fact-level consistency. Duplicate facts are merged (Step 5), but contradictory facts are linked by a CONTRADICTS edge rather than reconciled, and the reader must weigh them.

Abid: Reconcile with the fact-consolidation setting the runs used (Step 5 note).

Open-vocabulary relations. Relation labels are produced by the LLM under constraints rather than drawn from a fixed inventory such as the Penn Discourse Treebank's [35]. Synonymous labels (CAUSES, LEADS_TO) are matched only through their embeddings, and participation edges are unlabelled, so expansion out of an entity is not query-guided. Normalizing labels into a shared vocabulary, as ontology induction over UNH graphs does [29], would also make the index usable as a Semantic Web resource.

Scope. UNH targets questions whose answer follows a chain of specific statements. It does not target questions about a corpus as a whole ("What are the main themes of these policies?"), for which community summaries are designed [6]. The two are complementary: summaries could be written over UNH communities and kept alongside the edges. All systems share one LLM and one embedding model, and the multi-hop comparison uses 250 questions per dataset; small differences should be read against the bootstrap intervals.

6 CONCLUSION

Graph-based RAG indices connect knowledge only where facts mention the same entity, while many questions follow relations between statements: a cause and its effect, a rule and its exception, a claim and its rebuttal. We presented the Unified Narrative Hypergraph, an index that combines an n-ary fact hypergraph and a discourse graph over the same facts. Every atomic fact is both a hyperedge over its participants and a node in a graph of typed relations between facts, and each fact is traceable to its passage. Neither structure is new; their combination is what lets retrieval move between dependent facts that share no entity, choose its next step by the kind of relation a question asks for, and give the reader compact chains of grounded facts instead of passages or generated summaries. Construction stays linear in corpus size by judging relations within communities of a question-based fact-affinity graph rather than between all pairs. We compared UNH with six graph-based RAG systems under a protocol that fixes the language model, embedding model, reader, and judge, tested whether both structures are needed by comparing the hypergraph and discourse views of the same index, and reported every system's construction cost and reader context.

Abid: Add one sentence with the main empirical findings (RQ1–RQ4).

REFERENCES

[1] Haonan Bian. 2025. LLM-empowered knowledge graph construction: A survey. *arXiv preprint arXiv:2510.20345* (2025).

[2] Jaime Carbonell and Jade Goldstein. 1998. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st annual international ACM SIGIR conference on Research and development in information retrieval*. 335–336.

[3] Shengyuan Chen, Chuang Zhou, Zheng Yuan, Qinggang Zhang, Zeyang Cui, Hao Chen, Yilin Xiao, Jiannong Cao, and Xiao Huang. 2026. You Don't Need Pre-built Graphs for RAG: Retrieval Augmented Generation with Adaptive Reasoning Structures. In *Proceedings of the AAAI Conference on Artificial Intelligence*. arXiv:2508.06105.

[4] Tong Chen, Hongwei Wang, Sihao Chen, Wenhao Yu, Kaixin Ma, Xinran Zhao, Hongming Zhang, and Dong Yu. 2024. Dense X Retrieval: What Retrieval Granularity Should We Use?. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 15159–15177.

[5] Maxime Delmas, Lei Xu, and André Freitas. 2026. A Navigational Approach for Comprehensive RAG via Traversal over Proposition Graphs. *arXiv preprint arXiv:2601.04859* (2026).

[6] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024).

[7] Paul Groth, Andrew Gibson, and Jan Velterop. 2010. The Anatomy of a Nanopublication. *Information Services & Use* 30, 1–2 (2010), 51–56.

[8] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2024. LightRAG: Simple and Fast Retrieval-Augmented Generation. *arXiv preprint arXiv:2410.05779* (2024).

[9] Bernal J Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. Hipporag: Neurobiologically inspired long-term memory for large language models. *Advances in neural information processing systems* 37 (2024), 59532–59569.

[10] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. From RAG to Memory: Non-Parametric Continual Learning for Large Language

Models. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*.

[11] Haoyu Han, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A Rossi, Subhabrata Mukherjee, Xianfeng Tang, et al. 2024. Retrieval-augmented generation with graphs (graphrag). *arXiv preprint arXiv:2501.00309* (2024).

[12] Olaf Hartig. 2017. Foundations of RDF* and SPARQL* (An Alternative Approach to Statement-Level Metadata in RDF). In *Proceedings of the 11th Alberto Mendelzon International Workshop on Foundations of Data Management (AMW) (CEUR Workshop Proceedings, Vol. 1912)*.

[13] Yanning Hou, Duanyang Yuan, Sihang Zhou, Xiaoshu Chen, Ke Liang, Siwei Wang, Xinwang Liu, and Jian Huang. 2026. AtomicRAG: Atom-Entity Graphs for Retrieval-Augmented Generation. *arXiv preprint arXiv:2604.20844* (2026).

[14] Hao Hu et al. 2026. Cog-RAG: Cognitive-Inspired Dual-Hypergraph with Theme Alignment Retrieval-Augmented Generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[15] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *The Twelfth International Conference on Learning Representations*.

[16] Yassir Lairgi, Ludovic Moncla, Rémy Cazabet, Khalid Benabdeslem, and Pierre Cléau. 2024. itext2kg: Incremental knowledge graphs construction using large language models. In *International Conference on Web Information Systems Engineering*. Springer, 214–229.

[17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[18] Ruosen Li, Zimu Wang, Son Quoc Tran, Lei Xia, and Xinya Du. 2024. MEQA: A Benchmark for Multi-hop Event-centric Question Answering with Explanations. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

[19] Shilong Li, Yancheng He, Hangyu Guo, Xingyuan Bu, Ge Bai, Jie Liu, Jiaheng Liu, Xingwei Qu, Yangguang Li, Wanli Ouyang, Wenbo Su, and Bo Zheng. 2024. GraphReader: Building Graph-based Agent to Enhance Long-Context Abilities of Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 12758–12786.

[20] Wen-Sheng Lien, Yu-Kai Chan, Hao-Lung Hsiao, Bo-Kai Ruan, Meng-Fen Chiang, Chien-An Chen, Yi-Ren Yeh, and Hong-Han Shuai. 2026. HyperRAG: Reasoning N-ary Facts over Hypergraphs for Retrieval Augmented Generation. In *Proceedings of the ACM Web Conference 2026*. https://doi.org/10.1145/3774904.3792710 arXiv:2602.14470.

[21] Dongqi Liu, Hang Ding, Qiming Feng, Xurong Xie, Zhucun Xue, Chengjie Wang, Jian Li, Jiangning Zhang, and Yabiao Wang. 2026. Disco-RAG: Discourse-Aware Retrieval-Augmented Generation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 4106–4136.

[22] Haoran Luo, Haihong E, Guanting Chen, Yandan Zheng, Xiaobao Wu, Yikai Guo, Qika Lin, Yu Feng, Zemin Kuang, Meina Song, Yifan Zhu, and Luu Anh Tuan. 2025. HyperGraphRAG: Retrieval-Augmented Generation via Hypergraph-Structured Knowledge Representation. In *Advances in Neural Information Processing Systems*. arXiv:2503.21322.

[23] Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. 2023. Reasoning on graphs: Faithful and interpretable large language model reasoning. *arXiv preprint arXiv:2310.01061* (2023).

[24] Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huaren Qu, Cehao Yang, Jiaxin Mao, and Jian Guo. 2024. Think-on-graph 2.0: Deep and faithful large language model reasoning with knowledge-guided retrieval augmented generation. *arXiv preprint arXiv:2407.10805* (2024).

[25] William C. Mann and Sandra A. Thompson. 1988. Rhetorical structure theory: Toward a functional theory of text organization. *Text* 8, 3 (1988), 243–281.

[26] Costas Mavromatis and George Karypis. 2024. Gnn-rag: Graph neural retrieval for large language model reasoning. *arXiv preprint arXiv:2405.20139* (2024).

[27] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2023. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 12076–12100.

[28] Belinda Mo, Kyssen Yu, Joshua Kazdan, Joan Cabezas, Proud Mpala, Lisa Yu, Chris Cundy, Charilaos Kanatsoulis, and Sanmi Koyejo. 2025. Kggen: Extracting knowledge graphs from plain text with language models. *arXiv preprint arXiv:2502.09956* (2025).

[29] Maruf Ahmed Mridul, Abid Talukder, and Oshani Seneviratne. 2026. GrOIL: Graph-Grounded Domain Ontology Induction with Constrained LLM Mediation. In *Proceedings of the 35th ACM International Conference on Information and Knowledge Management (CIKM '26)*. https://doi.org/10.1145/3799682.3840910

[30] Rodrigo Nogueira and Kyunghyun Cho. 2019. Passage Re-ranking with BERT. *arXiv preprint arXiv:1901.04085* (2019).

[31] Natasha Noy and Alan Rector. 2006. Defining N-ary Relations on the Semantic Web. W3C Working Group Note. https://www.w3.org/TR/swbp-n-aryRelations/

[32] Pranoy Panda, Ankush Agarwal, Chaitanya Devaguptapu, Manohar Kaul, and Prathosh A P. 2024. HOLMES: Hyper-Relational Knowledge Graphs for Multi-hop Question Answering using LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 13263–13282.

[33] Richard Yuanzhe Pang, Alicia Parrish, Nitish Joshi, Nikita Nangia, Jason Phang, Angelica Chen, Vishakh Padmakumar, Johnny Ma, Jana Thompson, He He, and Samuel R. Bowman. 2022. QuALITY: Question Answering with Long Input Texts, Yes!. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 5336–5358.

[34] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2025. Graph retrieval-augmented generation: A survey. *ACM Transactions on Information Systems* 44, 2 (2025), 1–52.

[35] Rashmi Prasad, Bonnie Webber, and Aravind Joshi. 2017. The Penn Discourse Treebank: An annotated corpus of discourse relations. In *Handbook of linguistic annotation*. Springer, 1197–1217.

[36] Hongjin Qian, Peitian Zhang, Zheng Liu, Kelong Mao, and Zhicheng Dou. 2024. MemoRAG: Moving towards Next-Gen RAG via Memory-Inspired Knowledge Discovery. *arXiv preprint arXiv:2409.05591* (2024).

[37] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D Manning. 2024. Raptor: Recursive abstractive processing for tree-organized retrieval. In *The Twelfth International Conference on Learning Representations*.

[38] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M Ni, Heung-Yeung Shum, and Jian Guo. 2023. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. *arXiv preprint arXiv:2307.07697* (2023).

[39] Qingyun Sun, Jiaqi Yuan, Shan He, Xiao Guan, Haonan Yuan, Xingcheng Fu, Jianxin Li, and Philip S. Yu. 2025. DyG-RAG: Dynamic Graph Retrieval-Augmented Generation with Event-Centric Reasoning. *arXiv preprint arXiv:2507.13396* (2025).

[40] Vincent A. Traag, Ludo Waltman, and Nees Jan van Eck. 2019. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports* 9, 1 (2019), 5233.

[41] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics* 10 (2022), 539–554.

[42] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*. 10014–10037.

[43] Prakhar Verma, Sukruta Prakash Midigeshi, Gaurav Sinha, Arno Solin, Nagarajan Natarajan, and Amit Sharma. 2024. Plan*RAG: Efficient Test-Time Planning for Retrieval Augmented Generation. *arXiv preprint arXiv:2410.20753* (2024).

[44] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: A Free Collaborative Knowledgebase. *Commun. ACM* 57, 10 (2014), 78–85.

[45] Jingjin Wang and Jiawei Han. 2025. PropRAG: Guiding Retrieval with Beam Search over Proposition Paths. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 6212–6227.

[46] Nengbo Wang, Xiaotian Han, Jagdip Singh, Jing Ma, and Vipin Chaudhary. 2025. CausalRAG: Integrating Causal Graphs into Retrieval-Augmented Generation. In *Findings of the Association for Computational Linguistics: ACL 2025*. 22680–22693.

[47] Chuanjie Wu, Zhishang Xiang, Yunbo Tang, Zerui Chen, Qinggang Zhang, and Jinsong Su. 2026. MemGraphRAG: Memory-based Multi-Agent System for Graph Retrieval-Augmented Generation. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. https://doi.org/10.1145/3770855.3818074 arXiv:2606.00610.

[48] Zhishang Xiang, Chuanjie Wu, Qinggang Zhang, Shengyuan Chen, Zijin Hong, Xiao Huang, and Jinsong Su. 2025. When to use Graphs in RAG: A Comprehensive Analysis for Graph Retrieval-Augmented Generation. *arXiv preprint arXiv:2506.05690* (2025).

[49] Tianyang Xu, Haojie Zheng, Chengze Li, Haoxiang Chen, Yixin Liu, Ruoxi Chen, and Lichao Sun. 2025. NodeRAG: Structuring Graph-based RAG with Heterogeneous Nodes. *arXiv preprint arXiv:2504.11544* (2025).

[50] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-Mem: Agentic Memory for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[51] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2369–2380.

[52] Zairun Yang, Yilin Wang, Zhengyan Shi, Yuan Yao, Lei Liang, Keyan Ding, Emine Yilmaz, Huajun Chen, and Qiang Zhang. 2025. EventRAG: Enhancing LLM Generation with Event Knowledge Graphs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 16967–16979.

[53] Bowen Zhang and Harold Soh. 2024. Extract, define, canonicalize: An llm-based framework for knowledge graph construction. In *Proceedings of the 2024 conference on empirical methods in natural language processing*. 9820–9836.

[54] Hongming Zhang, Xin Liu, Haojie Pan, Yangqiu Song, and Cane Wing-Ki Leung. 2020. ASER: A Large-scale Eventuality Knowledge Graph. In *Proceedings of The Web Conference 2020*. 201–211.

[55] Nan Zhang, Prafulla Kumar Choubey, Alexander Fabbri, Gabriel Bernadett-Shapiro, Rui Zhang, Prasenjit Mitra, Caiming Xiong, and Chien-Sheng Wu. 2025. SiReRAG: Indexing Similar and Related Information for Multihop Reasoning. In *International Conference on Learning Representations (ICLR)*.

[56] Ze Yu Zhang, Zitao Li, Yaliang Li, Bolin Ding, and Bryan Kian Hsiang Low. 2026. Respecting Temporal-Causal Consistency: Entity-Event Knowledge Graphs for Retrieval-Augmented Generation. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*.

[57] Haoyang Zhong, Yifei Sun, Antong Zhang, Chunping Wang, Lei Chen, and Yang Yang. 2026. A Unified Framework for Context-Aware and Relation-Aware Graph Retrieval-Augmented Generation. In *Proceedings of the ACM Web Conference 2026*. https://doi.org/10.1145/3774904.3792720

[58] Yingli Zhou, Yaodong Su, Youran Sun, Shu Wang, Taotao Wang, Runyuan He, Yongwei Zhang, Sicong Liang, Xilin Liu, Yuchi Ma, and Yixiang Fang. 2025. In-depth Analysis of Graph-based RAG in a Unified Framework. *Proceedings of the VLDB Endowment* 18, 13 (2025), 5623–5637.

[59] Luyao Zhuang, Shengyuan Chen, Yilin Xiao, Huachi Zhou, Yujing Zhang, Hao Chen, Qinggang Zhang, and Xiao Huang. 2025. LinearRAG: Linear Graph Retrieval Augmented Generation on Large-scale Corpora. *arXiv preprint arXiv:2510.10114* (2025).

A FULL-SET BASELINE RESULTS

Table 5 reports the baselines on all 1,000 questions of each dataset, with the same reader and judge as Table 3. Contain-Acc is the fraction of answers that contain the normalized gold answer (the Str-Acc of MemGraphRAG [47] and LinearRAG [59]). Subset F1 lies within 4.2 points of full-set F1 for every system, inside the subset's sampling uncertainty, and the rankings agree except for one adjacent swap per dataset (GraphRAG and HyGRAG on HotpotQA; HippoRAG 2 and GraphRAG on MuSiQue).

B HYPERPARAMETERS

Table **??** lists the values used in all experiments. Symbols follow Section 3.

Abid: Values updated to the current run defaults (`evaluation_scripts/unh/main.py`): M 40→50, K 9→12, D 3→4, beam MMR 0.6→0.75. Confirm against the final runs, including the seed MMR λ and the 15 entity-resolution neighbours, which were not re-checked.

Table 5: Baselines on the full 1,000-question sets (shared reader and judge).

Method	HotpotQA				MuSiQue			
	EM	F1	Contain-Acc	LLM-Acc	EM	F1	Contain-Acc	LLM-Acc
HippoRAG 2	61.3	73.5	70.6	82.8	39.8	50.6	43.9	53.9
LogicRAG	58.2	72.8	70.2	82.9	35.8	48.5	42.3	52.8
MemGraphRAG	51.2	62.6	60.9	70.6	30.9	40.5	34.4	42.9
HyperGraphRAG	61.4	74.9	74.5	86.4	44.5	56.4	51.7	62.1
MS GraphRAG	56.4	69.2	67.1	79.6	38.2	49.1	42.9	54.2
HyGRAG	54.1	67.5	64.8	78.6	34.1	44.5	37.1	47.7